Contents

Symptom Tracker System — Study Guide

A working reference for how this project is built: what technology runs each layer, how the pieces talk to each other, and how to work with the database. Ends with a bank of progress-review questions and answers, split by layer.

---

1. What the system is

A school management web app for OKI International School, built to track student symptoms, emotional check-ins, break-time activities, study modules, doctor's recommendations, and messaging across five roles: **admin**, **branch principal**, **shadow teacher / class teacher**, **parent**, and **child**. Each role gets its own dashboard and permissions.

It's a classic three-tier architecture:

```
Browser (Next.js frontend)  <-- REST/JSON over HTTP -->  Express backend  <-- Mongoose -->  MongoDB
```

---

2. Tech stack at a glance

Layer	Technology	Version (from package.json)
Frontend framework	Next.js (App Router)	16.2.4
UI library	React	19.2.4
Language	TypeScript	^5
Styling	Tailwind CSS	^4
Charts	Recharts	^3.9.2
Backend runtime	Node.js (ES modules, `"type": "module"`)	—
Backend framework	Express	^5.2.1
Database	MongoDB	—
ODM (object-document mapper)	Mongoose	^9.7.0
Auth	JSON Web Tokens (`jsonwebtoken`) + `bcrypt` for password hashing	^9.0.3 / ^6.0.0

Layer	Technology	Version (from package.json)
File uploads	Multer	^2.0.0
Email	Nodemailer	^9.0.3
Dev tooling	nodemon (backend hot-reload), ESLint	—

Note: `mysql2` appears in `backend/package.json` but isn't used anywhere in the codebase — the project exclusively uses MongoDB/Mongoose. It's a leftover dependency, safe to ignore (or remove later).

---

3. Project folder structure

```
symptom-tracker-system/
├── backend/
│   ├── index.js              # Express app entry point — mounts all routes
│   ├── config/db.js          # MongoDB connection (mongoose.connect)
│   ├── models/                # Mongoose schemas (one file per collection)
│   ├── controllers/           # Business logic — one file per feature area
│   ├── routes/                # Express routers — map URLs to controller functions
│   ├── middleware/authMiddleware.js   # JWT verification + role gating
│   ├── utils/                 # Shared helpers (email, credentials, thresholds...)
│   └── uploads/                  # Uploaded files land here (served through auth-
checked routes, not statically)
│
└── frontend/
    ├── app/
    │   ├── login/, forgot-password/, forgot-username/, reset-password/
    │   └── dashboard/
    │       ├── admin/          # Admin-only pages
    │       ├── principal/      # Branch principal pages
    │       ├── teacher/        # Shadow/class teacher pages
    │       ├── parent/, child/ # Family-facing pages
    │       └── print-report/
    ├── components/           # Shared React components (layouts, Avatar, panels...)
    └── lib/                     # config.ts (API base URL), validation.ts, etc.
```

Rule of thumb: every backend feature has 3 files that work together — a **model** (what the data looks like), a **controller** (what happens when a request comes in), and a **route** (which URL + HTTP method triggers that controller).

---

4. Frontend: how it works

4.1 Next.js App Router basics

- Every folder under `app/` is a URL segment. `app/dashboard/teacher/students/[studentId]/page.tsx` maps to `/dashboard/teacher/students/<any-id>` — the square-bracket folder is a **dynamic route parameter**.

- Every page file starts with `"use client"` because these pages use React state/hooks (`useState`, `useEffect`) and browser APIs (`localStorage`) — they're not static server-rendered pages.
- `page.tsx` is what renders for a route. Components that aren't full pages live in `frontend/components/` and get imported.

4.2 Talking to the backend

There's no separate data-fetching library — pages use the browser's native `fetch`, wrapped around a shared constant:

```
// frontend/lib/config.ts
export const API_ORIGIN = process.env.NEXT_PUBLIC_API_BASE_URL || "http://localhost:5000";
export const API_BASE = `${API_ORIGIN}/api`;
```

A typical page load looks like this pattern, repeated across almost every dashboard page:

```
const [data, setData] = useState(null);
const [loading, setLoading] = useState(true);

useEffect(() => {
  const load = async () => {
    const token = localStorage.getItem("token");
    const res = await fetch(`${API_BASE}/students`, {
      headers: { Authorization: `Bearer ${token}` },
    });
    const json = await res.json();
    if (json.success) setData(json.students);
    setLoading(false);
  };
  load();
}, []);
```

Key points: - The JWT is stored in `localStorage` after login and attached to every request as `Authorization: Bearer <token>`. - Every backend response has a `success: boolean` — the frontend always checks it before trusting the payload.

4.3 Shared layout components

Each role has its own layout wrapper (`DashboardLayout`, `TeacherDashboardLayout`, `PrincipalDashboardLayout`, `FamilyDashboardLayout`) that renders the sidebar nav, header, notification bell, and user menu, then wraps whatever page content is passed as `children`. Pages just do:

```
return (
  <DashboardLayout>
    {/* page content */}
  </DashboardLayout>
);
```

4.4 Recurring UI patterns worth knowing

- **Avatar component** (`components/Avatar.tsx`) — deterministic colored circle with initials, used on every profile-style page header so there's a visual anchor.
- **Tabbed panels** — a `type Tab = "a" | "b" | "c"`, a `TABS` array of `{key, label}`, and a `tab` state var switched by clicking buttons; used for things like the teacher's Modules/Past Papers/Break-Time-Activities page and `MessagesPanel`.

- **Two-column layout** for profile pages: `grid grid-cols-1 lg:grid-cols-3 gap-6` — a `lg:col-span-2` main content column plus a `lg:col-span-1 lg:sticky lg:top-6` sidebar for secondary actions.
- **BackButton** — shared component used at the top of nearly every non-dashboard-home page for `router.back()` navigation.
- Forms follow a consistent shape: `useState` per field → `handleSubmit` does `fetch(..., { method: "POST"/"PATCH", body: JSON.stringify(...) })` → checks `data.success` → shows an error or success message.

4.5 Styling

Tailwind CSS utility classes are used directly in JSX (no separate CSS files per component, aside from a couple of legacy classes in `app/globals.css` for the login screen). Colors follow a light convention: `blue-900` for primary actions/headers, `emerald`/`green` for success, `red`/`amber` for errors/warnings, `gray` for neutral text.

---

5. Backend: how it works

5.1 Request lifecycle

`backend/index.js` is the entry point. It: 1. Loads environment variables (`dotenv.config()`) 2. Connects to MongoDB (`connectDB()`) 3. Creates the Express app, enables CORS and JSON body parsing 4. Mounts one router per feature area under `/api/...`:

```
app.use("/api/auth", authRoutes);
app.use("/api/students", studentRoutes);
app.use("/api/teacher", teacherRoutes);
app.use("/api/principal", principalRoutes);
app.use("/api/staff", staffRoutes);
app.use("/api/messages", messageRoutes);
app.use("/api/study-modules", studyModuleRoutes);
app.use("/api/doctor-documents", doctorDocumentRoutes);
app.use("/api/notices", noticeRoutes);
```

A request flows: **route** (matches URL + method) → **middleware** (auth checks) → **controller** (does the work, talks to the database, sends the response).

5.2 Authentication & authorization

Two middleware functions, defined once in `backend/middleware/authMiddleware.js`, are reused everywhere:

```
export const protect = (req, res, next) => {
  // reads "Authorization: Bearer <token>", verifies it with jwt.verify(),
  // and attaches the decoded payload as req.user = { id, role, branch }
};

export const authorizeRoles = (...allowedRoles) => {
  // returns a middleware that 403s unless req.user.role is in allowedRoles
};
```

Applied in routes like this:

```
router.use(protect);
router.use(authorizeRoles("admin"));
router.post("/principals", createPrincipal);
```

Login itself (`authController.js`) does the classic flow: look up the user by username, compare the submitted password against the stored hash with `bcrypt.compare`, then sign a JWT containing `{ id, role, branch }` with a 1-day expiry:

```
const match = await bcrypt.compare(password, user.password);
const token = jwt.sign({ id: user._id, role: user.role, branch: user.branch }, process.env.JWT_
```

Passwords are **never** stored in plain text — `bcrypt.hash(password, 10)` runs before any `User` document is saved.

5.3 Controller pattern

Controllers are plain async Express handlers, always wrapped in try/catch, always returning a consistent `{ success, message, ...data }` shape:

```
export const getPrincipals = async (req, res) => {
  try {
    const principals = await User.find({ role: "principal" }).select("name username branch emai
    res.json({ success: true, principals });
  } catch (error) {
    console.error(error);
    res.status(500).json({ success: false, message: "Server Error" });
  }
};
```

5.4 Environment variables

The backend expects a `.env` file (not committed to source control) with at least: - `MONGO_URI` — the MongoDB connection string - `JWT_SECRET` — secret key used to sign/verify tokens - `PORT` — defaults to 5000 if unset - SMTP credentials for Nodemailer (used by `backend/utils/mailer.js` to send account credentials/notifications)

5.5 File uploads

Handled with Multer. Uploaded files (doctor documents, study module resources, message attachments) are saved into `backend/uploads/` but **deliberately not** exposed as a static file server — each file type has its own authenticated download route (e.g. `GET /api/doctor-documents/:id/file`) that re-checks the requester's permissions before streaming the file back.

---

6. Database: MongoDB + Mongoose

MongoDB is a **document database** — instead of tables and rows (like MySQL), it has **collections** and **documents** (JSON-like objects). Mongoose sits on top of MongoDB and lets you define a **schema** (the shape a document must follow) and a **model** (the object you actually use to query/create/update/delete documents).

6.1 Connecting

```
// backend/config/db.js
import mongoose from "mongoose";
await mongoose.connect(process.env.MONGO_URI);
```

6.2 "Making a table" — defining a schema + model

This is the Mongoose equivalent of `CREATE TABLE`. Every model in this project follows the same shape:

```
import mongoose from "mongoose";

const studentSchema = new mongoose.Schema(
  {
    branch: { type: String, enum: BRANCHES, required: true },
    firstName: { type: String, required: true, trim: true },
    parentEmail: {
      type: String,
      required: true,
      lowercase: true,
      match: [/^[^\s@]+@[^\s@]+\.[^\s@]+$/, "Please provide a valid email address"],
    },
    assignedTeacher: { type: mongoose.Schema.Types.ObjectId, ref: "User", default: null }, // r
    flagged: { type: Boolean, default: false },
  },
  { timestamps: true } // auto-adds createdAt / updatedAt
);

const Student = mongoose.model("Student", studentSchema);
export default Student;
```

Things to notice: - `type`, `required`, `default`, `enum`, `unique`, `trim`, `match`/`validate` are all **schema-level constraints** — Mongoose enforces them before anything reaches MongoDB. - `ref: "User"` + `mongoose.Schema.Types.ObjectId` is how relationships are modeled — a Student document stores just the `_id` of a User document, and Mongoose can "join" them on read via `.populate()`. - `{ timestamps: true }` auto-manages `createdAt`/`updatedAt`.

Collections currently defined (`backend/models/`): `User`, `Student`, `TeacherProfile`, `SymptomLog`, `EmotionCheckin`, `BreakActivityLog`, `StudyResource`, `DoctorDocument`, `Message`, `Notice`, `Alert`.

6.3 Create

```
const principal = new User({ username, password: hashedPassword, role: "principal", name, branc
await principal.save();

// or, equivalently, in one call:
const doc = await TeacherProfile.create({ user: teacherUser._id, age, qualification });
```

6.4 Read

```
// Find one document matching a filter
const user = await User.findOne({ username });

// Find a document by its _id
```

```
const student = await Student.findById(studentId);

// Find many, with a filter
const teachers = await User.find({ role: "shadow_teacher" });

// Only return specific fields (like SELECT col1, col2 in SQL)
const principals = await User.find({ role: "principal" }).select("name username branch email cr

// Follow a ref relationship (like a SQL JOIN)
const students = await Student.find()
  .populate("assignedTeacher", "name username")
  .sort({ createdAt: -1 });

// Count documents matching a filter
const total = await Student.countDocuments({});

// Get distinct values of one field
const branchStudentIds = await Student.find({ branch }).distinct("_id");
```

6.5 Update

Two common styles used in this codebase:

```
// Style A — load it, mutate fields, save (used when you need validation/side-effects)
const principal = await User.findOne({ _id: id, role: "principal" });
principal.name = name.trim();
principal.email = trimmedEmail || null;
await principal.save();

// Style B — one-shot atomic update
const alert = await Alert.findByIdAndUpdate(
  id,
  { acknowledged: true, acknowledgedBy: req.user.id, acknowledgedAt: new Date() },
  { new: true } // return the updated document, not the old one
);
```

6.6 Delete

```
await SomeModel.findByIdAndDelete(id);
// or
await SomeModel.deleteOne({ _id: id });
```

(This project tends to favor soft-deletes/status flags over hard deletes for most user-facing data, but the underlying calls are as above where used.)

6.7 Querying rules of thumb

- Filters go in `{}` as the first argument: `Model.find({ role: "admin" })`.
- Mongoose queries are **promises** — always `await` them (or `.then()`), always wrap in `try/catch`.
- `.select("field1 field2")` limits which fields come back (leave out `password`, etc. for security).
- `.populate("fieldName", "fields to bring back")` resolves an `ObjectId` reference into the actual referenced document.
- `.sort({ field: 1 | -1 })`, `.limit(n)` chain onto any find.

---

7. Running the project locally

Backend:

```
cd backend
npm install
npm run dev     # nodemon index.js — auto-restarts on file changes
```

Frontend:

```
cd frontend
npm install
npm run dev      # next dev — runs on http://localhost:3000 by default
```

The backend defaults to port 5000; the frontend's `API_BASE` points at `http://localhost:5000/api` unless `NEXT_PUBLIC_API_BASE_URL` is set otherwise (needed once either side is deployed somewhere other than localhost).

---

8. Progress review — questions & answers

Frontend

Q1. What framework is the frontend built on, and what routing model does it use? A. Next.js (App Router). Routes are defined by folder structure under `app/` — a folder becomes a URL segment, and `[paramName]` folders create dynamic route parameters (e.g. `[studentId]`).

Q2. Why do most page files start with `"use client"`? A. Because they use React hooks (`useState`, `useEffect`) and browser-only APIs like `localStorage` to read the auth token — these require client-side rendering, not Next.js's default server components.

Q3. How does the frontend authenticate its API requests? A. On login, the backend returns a JWT, which is saved to `localStorage`. Every subsequent `fetch` call attaches it as an `Authorization: Bearer <token>` header.

Q4. How is the backend's URL configured, and why does that matter? A. Through a single constant, `API_BASE` in `frontend/lib/config.ts`, built from `NEXT_PUBLIC_API_BASE_URL` (or defaulting to `localhost:5000`). Centralizing it means the app can point at a different backend (staging, production) just by changing one environment variable instead of every fetch call.

Q5. How does the app avoid duplicating the sidebar/header on every page? A. Shared layout components (`DashboardLayout`, `TeacherDashboardLayout`, `PrincipalDashboardLayout`, `FamilyDashboardLayout`) wrap page-specific content passed in as `children`.

Q6. What styling approach is used, and why? A. Tailwind CSS utility classes written directly in JSX — no separate stylesheet per component. This keeps styling co-located with markup and avoids CSS naming/specificity issues.

Q7. How does a page know whether an API call succeeded? A. Every backend JSON response includes a `success: boolean` field; the frontend always checks `data.success` before using the returned data or shows `data.message` on failure.

Q8. Give an example of a reusable UI pattern in this app and why it was built as a shared component. A. The `Avatar` component — a deterministic, color-coded initials circle — is reused across every profile-style page (student, teacher, principal) so profile headers look

consistent and aren't just blocks of text, without duplicating the color/initials logic in every page.

Q9. How are multi-section pages (like Modules / Past Papers / Break-Time Activities) organized in the UI? A. With a tabbed interface: a `Tab` type, a `TABS` array of `{key, label}`, and a `tab` state variable that conditionally renders each section's content — the same pattern used by `MessagesPanel`.

Q10. What happens if a user's role doesn't match the dashboard they're viewing? A. The frontend routes by role after login (redirecting to `/dashboard/<role>`), but the actual enforcement happens on the backend — every protected endpoint re-checks `req.user.role` via `authorizeRoles(...)`, so a frontend-only restriction is not the real security boundary.

Backend

Q1. What web framework powers the backend, and what does `backend/index.js` do? A. Express. `index.js` is the app's entry point — it loads environment variables, connects to MongoDB, sets up CORS/JSON parsing middleware, and mounts one router per feature area (e.g. `/api/students`, `/api/auth`) before starting the HTTP server.

Q2. How does the backend verify who's making a request? A. The `protect` middleware (in `authMiddleware.js`) reads the `Authorization: Bearer <token>` header, verifies it with `jwt.verify()` using `JWT_SECRET`, and attaches the decoded payload (`{ id, role, branch }`) to `req.user` for downstream handlers to use.

Q3. How does the backend restrict an endpoint to specific roles? A. Via `authorize-Roles(...allowedRoles)`, a middleware factory chained after `protect` — e.g. `router.use(authorizeRoles("a` — that returns a 403 if `req.user.role` isn't in the allowed list.

Q4. How are passwords handled — are they ever stored as plain text? A. No. Passwords are hashed with `bcrypt.hash(password, 10)` before being saved to MongoDB, and on login, `bcrypt.compare(submittedPassword, storedHash)` checks the match without ever decrypting the stored hash.

Q5. What's the standard shape of a controller function in this codebase? A. An async Express handler wrapped in try/catch, returning JSON with a `success: boolean` plus either the requested data or an error `message`. On error, it logs the error server-side and responds with `res.status(500).json({ success: false, message: "Server Error" })`.

Q6. How is file upload handled, and why isn't `/uploads` just served statically? A. Multer handles multipart form uploads, saving files into `backend/uploads/`. The folder is intentionally not exposed via a static file route, because that would let anyone with a guessed/leaked URL download a file with no login check. Instead, each file type has its own authenticated route (e.g. `GET /api/doctor-documents/:id/file`) that re-runs the same ownership/permission check as the metadata endpoint before streaming the file.

Q7. How does the credential-emailing feature degrade if the mail server is unreachable? A. Account creation (student/parent/teacher/principal) always saves the database record first; the email send is wrapped in its own try/catch afterward, so a failed send never blocks account creation. The response includes an `emailSent: boolean` so the frontend can show accurate copy ("credentials emailed" vs. "please share these manually").

Q8. What are the three files every backend feature needs, and what's each one's job? A. A **model** (Mongoose schema defining the data shape), a **controller** (the function that runs when a request comes in — validates input, talks to the database, returns a response), and a **route** (maps an HTTP method + URL path to a controller function, with any middleware in between).

Q9. How does the backend know which branch/role context to scope a query to? A. From `req.user`, set by the `protect` middleware from the JWT payload — e.g. a principal's queries are filtered by `req.user.branch`, and role checks use `req.user.role`.

Q10. Why does the JWT include `role` and `branch` instead of the backend looking them up fresh on every request? A. Performance and simplicity — since the JWT is signed and only the server can generate a valid one, embedding `role`/`branch` avoids an extra database lookup on every request. The trade-off is that a role/branch change won't take effect until the user's token is refreshed (re-login), which is an accepted limitation here.

Database

Q1. What kind of database does this project use, and how is it different from a SQL database like MySQL? A. MongoDB, a document database. Instead of fixed-schema tables and rows joined by foreign keys, it stores flexible JSON-like documents in collections. Relationships are modeled by storing another document's `_id` (an `ObjectId`) as a reference field, rather than enforced foreign keys.

Q2. What is Mongoose, and why is it used instead of talking to MongoDB directly? A. Mongoose is an ODM (Object-Document Mapper) for MongoDB. It lets you define a schema (required fields, types, defaults, validation rules, enums) and gives you a Model with convenient methods (`find`, `create`, `save`, `populate`, etc.) instead of writing raw MongoDB driver calls by hand.

Q3. How do you define a new collection ("make a table") in this project? A. Create a new file in `backend/models/`, define a `new mongoose.Schema({...})` describing each field's type/constraints, then call `mongoose.model("CollectionName", schema)` and export it. Mongoose auto-creates the actual MongoDB collection (pluralized, lowercased) the first time a document is saved.

Q4. How do you represent a relationship between two collections, e.g. a Student and their assigned Teacher? A. Store the related document's `_id` using `{ type: mongoose.Schema.Types.ObjectId, ref: "User" }`. To "join" and pull in the actual teacher data when reading, call `.populate("assignedTeacher", "name username")` on the query.

Q5. Write the Mongoose call to find all shadow teachers, returning only their name and username. A. `await User.find({ role: "shadow_teacher" }).select("name username")`

Q6. What's the difference between `Model.find()` and `Model.findOne()`? A. `find()` returns an array of all matching documents (empty array if none match); `findOne()` returns a single document (the first match) or `null`.

Q7. How do you update an existing document, and what are the two common approaches used in this codebase? A. Either (1) fetch it with `findOne`/`findById`, mutate the fields in JS, then call `.save()` — useful when you need validation or conditional logic before saving — or (2) call `Model.findByIdAndUpdate(id, { fields }, { new: true })` for a single atomic update that also returns the updated document.

Q8. How does Mongoose enforce data integrity, e.g. making sure an email is valid or a `role` is one of a fixed set of values? A. Through schema-level constraints: `required: true`, `enum: [...]` (restrict to a fixed list of values), `unique: true` (no duplicates), and custom `validate`/`match` functions/regexes — all checked before a document is written to MongoDB.

Q9. How is a connection to the database established when the backend starts? A. `backend/config/db.js` exports `connectDB()`, which calls `await mongoose.connect(process.env.MONGO_URI)`

— the connection string is kept in an environment variable, not hardcoded, so it can point at different databases (local/staging/production) without code changes.

Q10. Name a few collections in this database and what each stores. A. `User` (login accounts for every role), `Student` (student profile + parent info + assigned teacher), `SymptomLog` and `EmotionCheckin` (day-to-day tracking data), `TeacherProfile` (extra fields for shadow teachers, linked to a `User`), `Alert` (auto/manual "needs attention" flags), `Message` (staff/parent messaging), `StudyResource` and `DoctorDocument` (uploaded file metadata).

Q11. Why does the `User` model store `role` and `branch` but not always require `branch`? A. `branch` only makes sense for roles tied to one school location — `principal` (the branch they manage) and `class_teacher` (who sees their branch's students). Other roles (`admin`, `shadow_teacher`, `parent`, `child`) either work across all branches or derive their branch indirectly (a shadow teacher's branch comes from whichever students they're assigned to, not a stored field), so `branch` is left optional at the schema level.

Q12. What does `{ timestamps: true }` do in a schema, and why is it useful? A. It automatically adds and maintains `createdAt` and `updatedAt` fields on every document, without any manual code — used throughout this project for sorting ("most recent first") and audit purposes.

---

9. Quick-reference cheat sheet

I want to…	Do this
Add a new database collection	Create a schema + model file in `backend/models/`
Add a new API endpoint	Add a controller function in `backend/controllers/`, then wire it to a route in `backend/routes/`
Restrict an endpoint to certain roles	`router.use(protect); router.use(authorizeRoles("admin", "principal"))`
Fetch data on a page	`useEffect` + `fetch(${API_BASE}/..., { headers: { Authorization: \Bearer ${token}‘ } })| | Add a new frontend page | Create apage.tsxunder the rightapp/dashboard//...folder | | Look up one document by ID |Model.findById(id)orModel.findOne({ _id: id, ...otherFilters })| | Pull in a related document |.populate(“fieldName”, “fields to include”)| | Update a document safely with validation | Load it, mutate fields,await doc.save()| | Send an account-related email |sendEmail({ to, subject, html })frombackend/utils/mailer.js‘, wrapped in try/catch